

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



ANDROID LAYOUT WITH COMPOSE

Oleh:

Muhammad Dzul Fathi Ahyan

NIM. 2410817210011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Dzul Fathi Ahyan
NIM : 2410817210011

Menyetujui,
Assisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Raymond Hariyono
NIM. 2310817210007

Erika Maulidiya, S.Kom., M.Kom
NIP. 20000192025062016

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI	iii
DAFTAR GAMBAR	iv
DAFTAR TABEL.....	v
SOAL 1	6
A. Source Code	8
Source Code XML	8
Source Code Compose	15
B. Output Program.....	22
C. Pembahasan.....	25
Pembahasan XML	25
Pembahasan Compose.....	31
SOAL 2	35
A. Pembahasan.....	35
SOAL 3	37
A. Pembahasan.....	37
DAFTAR PUSTAKA	38
TAUTAN GIT	39

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi	6
Gambar 2. Tampilan Pilihan Persentase Tip	7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan	7
Gambar 4. Tampilan Aplikasi XML.....	22
Gambar 5. Tampilan Dropdown Tip XML.....	23
Gambar 6. Tampilan Round Up Tip XML	23
Gambar 7. Tampilan Aplikasi Compose	24
Gambar 8. Tampilan Dropdown Tip Compose	24
Gambar 9. Tampilan Round Up Tip Compose.....	25

DAFTAR TABEL

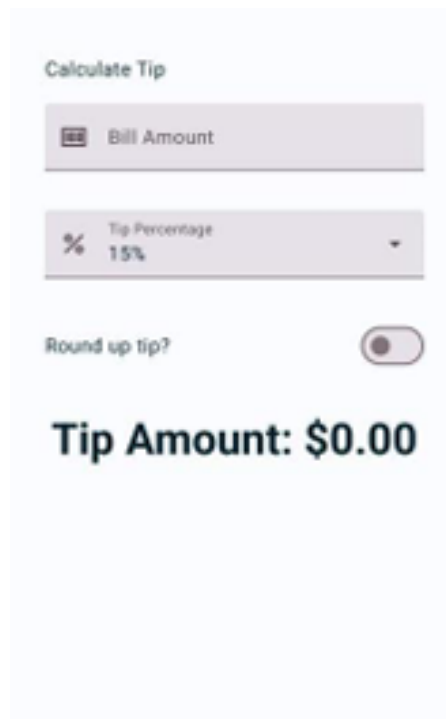
Tabel 1. Source Code Jawaban Soal 1 XML.....	8
Tabel 2. Source Code Jawaban Soal 1 XML.....	11
Tabel 3. Source Code Jawaban Soal 1 Compose	15
Tabel 4 Perbandingan Jawaban Soal 2	35

SOAL 1

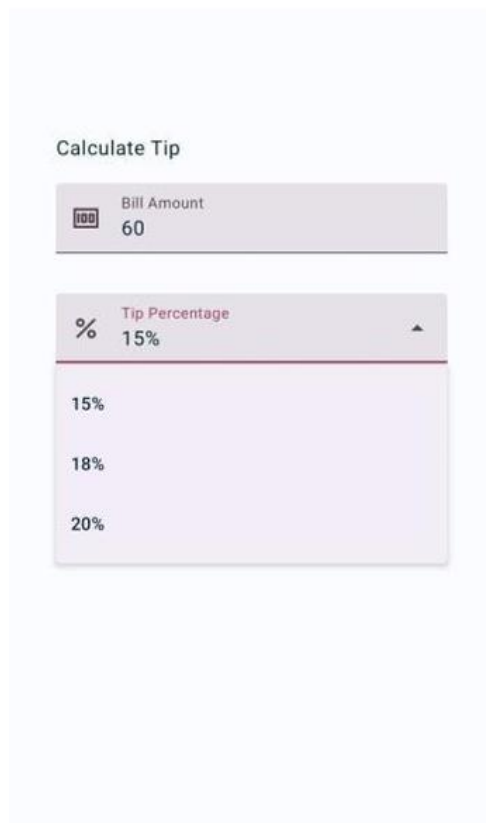
Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

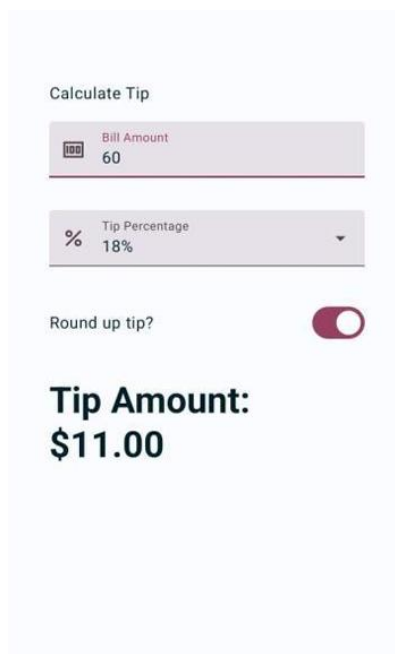
- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

Source Code XML

Activity_main.xml

Tabel 1. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res
	/android"
3	xmlns:app="http://schemas.android.com/apk/res-
	auto"
4	android:id="@+id/main"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:fillViewport="true">
8	
9	<LinearLayout
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:layout_gravity="center_vertical"
13	android:gravity="center_horizontal"
14	android:orientation="vertical"
15	android:padding="40dp">
16	
17	<TextView
18	android:layout_width="match_parent"
19	android:layout_height="wrap_content"
20	android:layout_marginBottom="16dp"
21	android:gravity="start"
22	android:text="Calculate Tip"
23	android:textSize="24sp" />
24	

25	<com.google.android.material.textfield.TextInputLayout
	layout
26	style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox"
27	android:layout_width="match_parent"
28	android:layout_height="wrap_content"
29	android:layout_marginBottom="16dp"
30	app:startIconDrawable="@android:drawable/ic_menu_edit">
31	
32	<com.google.android.material.textfield.TextInputEditText
33	android:id="@+id/etBillAmount"
34	android:layout_width="match_parent"
35	android:layout_height="wrap_content"
36	android:hint="Bill Amount"
37	android:inputType="numberDecimal"
38	android:singleLine="true" />
39	</com.google.android.material.textfield.TextInputLayout>
40	
41	<com.google.android.material.textfield.TextInputLayout
	layout
42	style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropDownMenu"
43	android:layout_width="match_parent"
44	android:layout_height="wrap_content"
45	android:layout_marginBottom="32dp"
46	app:prefixText="% ">
47	
48	<AutoCompleteTextView
49	android:id="@+id/dropdownTip"

50	android:layout_width="match_parent"
51	android:layout_height="wrap_content"
52	android:hint="Tip Percentage"
53	android:inputType="none" />
54	</com.google.android.material.textfield.TextInputLayout>
55	
56	<LinearLayout
57	android:layout_width="match_parent"
58	android:layout_height="wrap_content"
59	android:layout_marginBottom="32dp"
60	android:gravity="center_vertical"
61	android:orientation="horizontal">
62	
63	<TextView
64	android:layout_width="0dp"
65	android:layout_height="wrap_content"
66	android:layout_weight="1"
67	android:text="Round up tip?"
68	android:textSize="16sp" />
69	
70	<com.google.android.material.switchmaterial.SwitchMaterial
71	android:id="@+id/switchRoundUp"
72	android:layout_width="wrap_content"
73	android:layout_height="wrap_content" />
74	</LinearLayout>
75	
76	<TextView
77	android:id="@+id/tvTipAmount"
78	android:layout_width="wrap_content"
79	android:layout_height="wrap_content"

80	android:text="Tip Amount: \$0.00"
81	android:textSize="36sp"
82	android:textStyle="bold" />=
83	
84	</LinearLayout>
85	</ScrollView>

MainActivity.kt

Tabel 2. Source Code Jawaban Soal 1 XML

1	package com.example.modul2xml
2	
3	import android.os.Bundle
4	import android.text.Editable
5	import android.text.TextWatcher
6	import android.widget.AdapterView
7	import android.widget.AutoCompleteTextView
8	import android.widget.TextView
9	import androidx.activity.enableEdgeToEdge
10	import androidx.appcompat.app.AppCompatActivity
11	import androidx.core.view.ViewCompat
12	import androidx.core.view.WindowInsetsCompat
13	import
	com.google.android.material.switchmaterial.SwitchMaterial
14	import
	com.google.android.material.textfield.TextInputEditText
15	import java.text.NumberFormat
16	
17	class MainActivity : AppCompatActivity() {
18	

19	private lateinit var etBillAmount: TextInputEditText
20	private lateinit var dropdownTip: AutoCompleteTextView
21	private lateinit var switchRoundUp: SwitchMaterial
22	private lateinit var tvTipAmount: TextView
23	
24	
25	override fun onCreate(savedInstanceState: Bundle?) {
26	super.onCreate(savedInstanceState)
27	enableEdgeToEdge()
28	setContentView(R.layout.activity_main)
29	
30	ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
31	val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
32	v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
33	insets
34	}
35	
36	etBillAmount = findViewById(R.id.etBillAmount)
37	dropdownTip = findViewById(R.id.dropdownTip)

38	switchRoundUp	=
	findViewById(R.id.switchRoundUp)	
39	tvTipAmount	=
	findViewById(R.id.tvTipAmount)	
40		
41	val tipOptions = listOf("15", "18", "20")	
42	val adapter = ArrayAdapter(this,	
	android.R.layout.simple_list_item_1, tipOptions)	
43	dropdownTip.setAdapter(adapter)	
44	dropdownTip.setText(tipOptions[0], false)	
45		
46		
47		
	etBillAmount.addTextChangedListener(object :	
	TextWatcher {	
48	override fun beforeTextChanged(s:	
	CharSequence?, start: Int, count: Int, after: Int)	
	{}	
49	override fun onTextChanged(s:	
	CharSequence?, start: Int, before: Int, count: Int)	
	{}	
50	override fun afterTextChanged(s:	
	Editable?) {	
51	calculateAndDisplayTip()	
52	}	
53	})	
54		
55	dropdownTip.setOnItemClickListener { _, _,	
	_, _ ->	
56	calculateAndDisplayTip()	
57	}	
58		

59	switchRoundUp.setOnCheckedChangeListener
	{ _, _ ->
60	calculateAndDisplayTip()
61	}
62	}
63	
64	private fun calculateAndDisplayTip() {
65	val amountStr =
	etBillAmount.text.toString()
66	val tipStr = dropdownTip.text.toString()
67	
68	val amount = amountStr.toDoubleOrNull() ?:
	0.0
69	val tipPercent = tipStr.toDoubleOrNull()
	?: 0.0
70	val isRoundUp = switchRoundUp.isChecked
71	
72	var tip = (tipPercent / 100) * amount
73	if (isRoundUp) {
74	tip = kotlin.math.ceil(tip)
75	}
76	
77	val formattedTip =
	NumberFormat.getCurrencyInstance().format(tip)
78	tvTipAmount.text = "Tip Amount:
	\$formattedTip"
79	}
80	}

Source Code Compose

MainActivity.kt

Tabel 3. Source Code Jawaban Soal 1 Compose

1	package com.example.modul2compopse
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import
	androidx.compose.foundation.layout.Arrangement
8	import androidx.compose.foundation.layout.Column
9	import androidx.compose.foundation.layout.Row
10	import
	androidx.compose.foundation.layout.fillMaxSize
11	import
	androidx.compose.foundation.layout.fillMaxWidth
12	import androidx.compose.foundation.layout.padding
13	import
	androidx.compose.foundation.rememberScrollState
14	import
	androidx.compose.foundation.text.KeyboardOptions
15	import androidx.compose.foundation.verticalScroll
16	import androidx.compose.material.icons.Icons
17	import
	androidx.compose.material.icons.filled.Edit
18	import
	androidx.compose.material3.DropdownMenuItem
19	import
	androidx.compose.material3.ExperimentalMaterial3A pi

20	import androidx.compose.material3.ExposedDropDownMenuBox
21	import androidx.compose.material3.ExposedDropDownMenuDefaults
22	import androidx.compose.material3.Icon
23	import androidx.compose.material3.MaterialTheme
24	import androidx.compose.material3.Surface
25	import androidx.compose.material3.Switch
26	import androidx.compose.material3.Text
27	import androidx.compose.material3.TextField
28	import androidx.compose.runtime.Composable
29	import androidx.compose.runtime.getValue
30	import androidx.compose.runtime.mutableStateOf
31	import androidx.compose.runtime.remember
32	import androidx.compose.runtime.setValue
33	import androidx.compose.ui.Alignment
34	import androidx.compose.ui.Modifier
35	import androidx.compose.ui.text.input.KeyboardType
36	import androidx.compose.ui.unit.dp
37	import com.example.modul2compopse.ui.theme.Modul2Compops eTheme
38	import java.text.NumberFormat.getCurrencyInstance
39	
40	
41	class MainActivity : ComponentActivity() {
42	override fun onCreate(savedInstanceState: Bundle?) {
43	super.onCreate(savedInstanceState)
44	enableEdgeToEdge()

45	setContent {			
46	Modul2CompopseTheme {			
47	Surface(			
48	modifier	=		
	Modifier.fillMaxSize(),			
49	color	=		
	MaterialTheme.colorScheme.background			
50	) {			
51	TipCalculatorApp()			
52	}			
53	}			
54	}			
55	}			
56	}			
57				
58	@OptIn(ExperimentalMaterial3Api::class)			
59	@Composable			
60	fun TipCalculatorApp() {			
61	// untuk textfield			
62	var amountInput by remember {			
	mutableStateOf("") }			
63				
64	// untuk dropdown			
65	var tipInput by remember			
	{mutableStateOf("15") }			
66	var expanded by remember			
	{mutableStateOf(false) }			
67	val tipOptions = listOf("15","18","20")			
68				
69	var roundUp by remember {			
	mutableStateOf(false) }			
70				

71	val amount = amountInput.toDoubleOrNull() ?:
	0.0
72	val tipPercent = tipInput.toDoubleOrNull() ?:
	0.0
73	
74	val tip = calculateTip(amount, tipPercent,
	roundUp)
75	
76	Column(
77	modifier = Modifier
78	.padding(40.dp)
79	
	.verticalScroll(rememberScrollState()),
80	horizontalAlignment =
	Alignment.CenterHorizontally,
81	verticalArrangement = Arrangement.Center
82	) {
83	// Menampilkan teks judul
84	Text(
85	text = "Calculate Tip",
86	modifier = Modifier
87	.padding(bottom = 16.dp)
88	.align(Alignment.Start)
89	)
90	
91	TextField(
92	value = amountInput,
93	onValueChange = { newValue ->
94	if (newValue.isEmpty()
	newValue.toDoubleOrNull() != null) {
95	amountInput = newValue
96	}

```

97         },
98         label = { Text("Bill Amount") },
99         leadingIcon = {
100             Icon(
101                 imageVector
102                 Icons.Default.Edit,
103                 contentDescription = "Bill
104                 Icon"
105             )
106         },
107         singleLine = true,
108         keyboardOptions
109         KeyboardOptions(keyboardType
110         KeyboardType.Number),
111         modifier = Modifier
112             .fillMaxWidth()
113             .padding(bottom = 16.dp)
114     )
115
116     //dropdown tip
117     ExposedDropdownMenuBox(
118         expanded = expanded,
119         onExpandedChange = { expanded =
120         !expanded },
121         modifier = Modifier.padding(bottom =
122         32.dp)
123     ) {
124         TextField(
125             value = "$tipInput%",
126             onValueChange = { },
127             readOnly = true,

```

```

122         label = { Text("Tip Percentage")
123     },
124     leadingIcon = {
125         Text(
126             text = "%",
127             style =
128                 MaterialTheme.typography.titleMedium
129             )
130     },
131     trailingIcon = {
132         ExposedDropdownMenuDefaults.TrailingIcon(expanded
133         = expanded) },
134     colors =
135         ExposedDropdownMenuDefaults.textFieldColors(),
136     modifier = Modifier
137         .fillMaxWidth()
138         .menuAnchor()
139     )
140     ExposedDropdownMenu(
141         expanded = expanded,
142         onDismissRequest = { expanded =
143         false}
144     ) {
145         tipOptions.forEach {
146             selectionOption ->
147             DropdownMenuItem(
148                 text = {
149                     Text("$selectionOption%") },
150                 onClick = {
151                     tipInput =
152                     selectionOption
153                     expanded = false

```

```

145         }
146     )
147 }
148 }
149 }
150
151 Row(
152     modifier = Modifier
153         .fillMaxWidth()
154         .padding(bottom = 32.dp),
155     verticalAlignment =
Alignment.CenterVertically,
156     horizontalArrangement =
Arrangement.SpaceBetween
157 ) {
158     Text(text = "Round up tip?")
159     Switch(
160         checked = roundUp,
161         onCheckedChange = { roundUp = it }
162     )
163 }
164
165 Text(
166     text = "Tip Amount: $tip",
167     style =
MaterialTheme.typography.displaySmall
168 )
169 }
170 }
171
172 private fun calculateTip(amount: Double,
    tipPercent: Double, roundUp: Boolean): String {

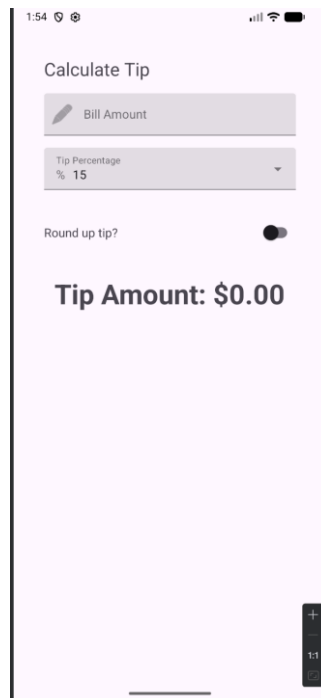
```

```

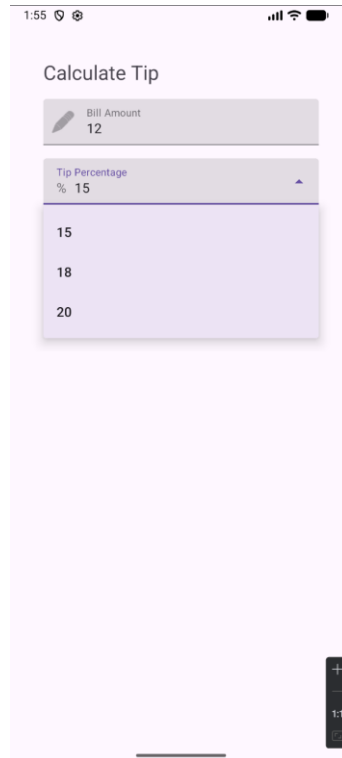
173
174     var tip = (tipPercent / 100) * amount
175
176
177     if (roundUp) {
178         tip = kotlin.math.ceil(tip)
179     }
180
181     return getCurrencyInstance().format(tip)
182 }

```

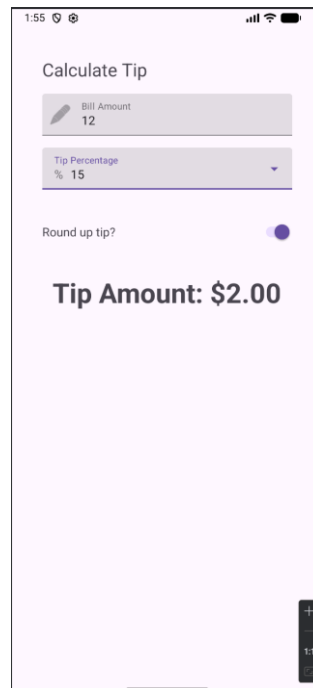
B. Output Program



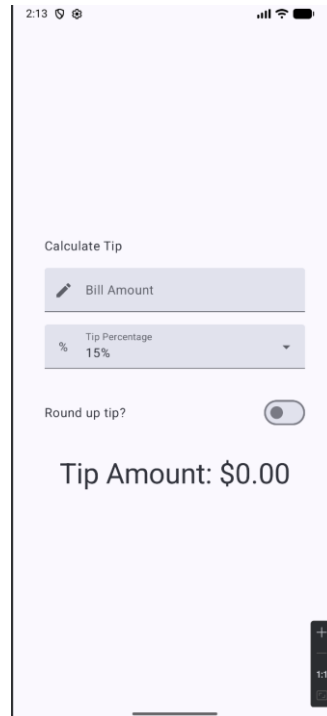
Gambar 4. Tampilan Aplikasi XML



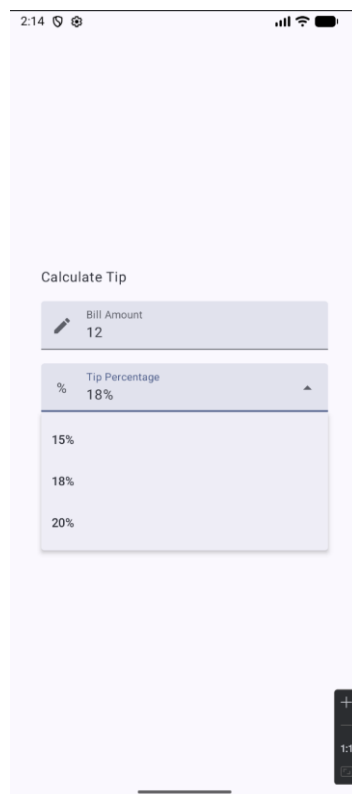
Gambar 5. Tampilan Dropdown Tip XML



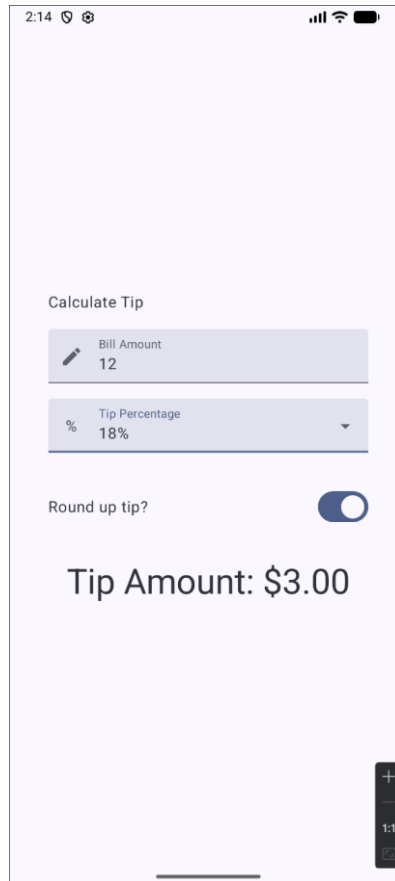
Gambar 6. Tampilan Round Up Tip XML



Gambar 7. Tampilan Aplikasi Compose



Gambar 8. Tampilan Dropdown Tip Compose



Gambar 9. Tampilan Round Up Tip Compose

C. Pembahasan

Pembahasan XML

MainActivity.kt:

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul2xml`. Ini menentukan identitas dari project aplikasi versi XML kamu.

Line 2: Baris kosong sebagai pemisah.

Line 3 - 15: Blok import yang memasukkan class dan library komponen UI tradisional dari sistem Android dan Material Design (seperti `TextView`, `SwitchMaterial`, dll). Kita juga mengimpor `AppCompatActivity` sebagai kelas dasar activity tradisional, serta `NumberFormat` untuk memformat mata uang.

Line 16: Baris kosong.

Line 17: Mendeklarasikan class `MainActivity` yang mewarisi (extends) `AppCompatActivity()`. Ini adalah pendekatan standar Android sebelum adanya Jetpack Compose.

Line 18: Baris kosong.

Line 19 - 22: Mendeklarasikan variabel-variabel komponen UI menggunakan modifier `private lateinit var`. Variabel ini (`etBillAmount`, `dropdownTip`, `switchRoundUp`, `tvTipAmount`) disiapkan untuk disambungkan dengan komponen di file XML nanti.

Line 23 - 24: Baris kosong.

Line 25: Meng-override fungsi `onCreate`. Ini adalah titik awal (entry point) di mana siklus hidup aplikasi berjalan saat pertama kali dibuka.

Line 26: Memanggil `super.onCreate` agar sistem Android memproses pembuatan activity bawaannya.

Line 27: Memanggil `enableEdgeToEdge()` agar tampilan aplikasi bisa memanjang penuh ke ujung layar (termasuk menembus area status bar).

Line 28: `setContentView(R.layout.activity_main)` adalah baris paling krusial di pendekatan XML. Fungsi inilah yang menyambungkan file Kotlin ini dengan file desain `activity_main.xml`.

Line 29: Baris kosong.

Line 30 - 34: Blok `ViewCompat.setOnApplyWindowInsetsListener` bertugas mengatur padding layar secara dinamis agar komponen UI tidak tertutup oleh bilah sistem (seperti poni kamera HP atau tombol navigasi bawah).

Line 35: Baris kosong.

Line 36: Menggunakan fungsi `findViewById` untuk mencari komponen input teks di file XML yang memiliki ID `etBillAmount`, lalu menyambungkannya ke variabel `etBillAmount`.

Line 37: Mencari komponen dropdown dengan ID `dropdownTip` dan menyambungkannya ke variabel `dropdownTip`.

Line 38: Mencari komponen switch dengan ID `switchRoundUp` dan menyambungkannya ke variabel `switchRoundUp`.

Line 39: Mencari komponen teks dengan ID `tvTipAmount` dan menyambungkannya ke variabel `tvTipAmount`.

Line 40: Baris kosong.

Line 41: Membuat daftar koleksi angka "15", "18", dan "20" lalu menyimpannya di variabel `tipOptions`.

Line 42: Membuat ArrayAdapter yang berfungsi untuk mengubah daftar teks tipOptions menjadi format baris pilihan yang dipahami oleh komponen dropdown.

Line 43: Memasang adapter yang baru dibuat tadi ke dalam komponen dropdownTip.

Line 44: Memanggil fungsi setText pada dropdownTip untuk menampilkan opsi pertama ("15") sebagai nilai bawaan (default) saat aplikasi pertama kali dibuka.

Line 45 - 46: Baris kosong.

Line 47 - 53: etBillAmount.addTextChangedListener { memasang pendeteksi ketikan pada kotak input biaya. Fungsi afterTextChanged di baris 50 akan memanggil calculateAndDisplayTip() setiap kali pengguna selesai mengetik atau menghapus angka.

Line 54: Baris kosong.

Line 55 - 57: dropdownTip.setOnItemClickListener { memasang pendeteksi ketukan. Jika salah satu angka persentase di dalam dropdown dipilih, maka fungsi calculateAndDisplayTip() akan dieksekusi.

Line 58: Baris kosong.

Line 59 - 61: switchRoundUp.setOnCheckedChangeListener { memasang pendeteksi perubahan status. Jika switch digeser menjadi nyala atau mati, maka fungsi calculateAndDisplayTip() akan dieksekusi.

Line 62: Penutup kurung kurawal untuk blok fungsi onCreate.

Line 63: Baris kosong.

Line 64: Mendeklarasikan fungsi tambahan bernama calculateAndDisplayTip() yang bersifat private. Fungsi inilah yang menangani seluruh logika matematika aplikasi.

Line 65: Mengambil teks yang sedang diketik pada etBillAmount, mengubahnya menjadi String, dan menyimpannya di variabel amountStr.

Line 66: Mengambil angka persentase yang sedang tampil pada dropdownTip, mengubahnya menjadi String, dan menyimpannya di variabel tipStr.

Line 67: Baris kosong.

Line 68: Mengonversi nilai huruf amountStr menjadi angka desimal (Double). Jika kotaknya kosong atau nilainya tidak valid, fungsi toDoubleOrNull() akan mengembalikan nilai aman yaitu 0.0 menggunakan operator elvis (?:).

Line 69: Melakukan konversi yang sama untuk tipStr menjadi angka desimal (Double).

Line 70: Mengecek apakah switch pembulatan sedang menyala (true) atau mati (false), lalu menyimpannya di variabel isRoundUp.

Line 71: Baris kosong.

Line 72: Menghitung nilai tip awal dengan membagi persentase (tipPercent) dengan 100, lalu dikalikan dengan total biaya (amount).

Line 73 - 75: Membuka kondisional if. Jika isRoundUp bernilai true, maka nilai tip akan dibulatkan ke atas menggunakan fungsi matematika kotlin.math.ceil().

Line 76: Baris kosong.

Line 77: Memanggil fungsi `getCurrencyInstance().format()` untuk menyisipkan simbol mata uang ke dalam angka tip secara otomatis, lalu menyimpannya di variabel `formattedTip`.

Line 78: Menggabungkan teks "Tip Amount: " dengan angka yang sudah diformat, lalu menampilkannya ke layar dengan memodifikasi properti text pada komponen `tvTipAmount`.

Line 79 - 80: Penutup kurung kurawal untuk fungsi `calculateAndDisplayTip()`, dan penutup untuk kelas `MainActivity`.

activity_main.xml:

Line 1: Merupakan deklarasi standar XML yang mendefinisikan versi XML dan format encoding (utf-8) yang digunakan dalam berkas ini.

Line 2 - 7: Membuka tag <ScrollView> sebagai wadah paling luar (root layout). Komponen ini memungkinkan layar untuk digulir (scroll) jika konten melebihi ukuran layar ponsel. Atribut android:fillViewport="true" di baris 7 sangat penting untuk memastikan area gulir membentang penuh menutupi seluruh tinggi layar.

Line 8: Baris kosong.

Line 9 - 15: Membuka tag <LinearLayout> dengan orientasi vertikal. Komponen ini berfungsi persis seperti Column pada Jetpack Compose, yaitu menyusun elemen-elemen di dalamnya dari atas ke bawah. Atribut android:layout_gravity="center_vertical" di baris 12 bertugas mendorong seluruh susunan form ini agar berada tepat di tengah layar.

Line 16: Baris kosong.

Line 17 - 23: Mendefinisikan <TextView> untuk menampilkan teks judul "Calculate Tip". Atribut android:gravity="start" memastikan teks ini rata kiri.

Line 24: Baris kosong.

Line 25 - 30: Membuka <TextInputLayout> dari library Material Design. Ini adalah kerangka luar untuk kotak input biaya. Atribut style="...FilledBox" memberikan gaya kotak abu-abu dengan garis bawah, dan app:startIconDrawable menambahkan ikon pensil di sebelah kiri kotak.

Line 31: Baris kosong.

Line 32 - 38: Mendefinisikan <TextInputEditText> sebagai area tempat pengguna mengetik nominal biaya. Komponen ini diberi ID etBillAmount agar bisa dipanggil di Kotlin. Atribut android:inputType="numberDecimal" memaksa papan ketik (keyboard) yang muncul hanya menampilkan angka.

Line 39: Penutup untuk blok kerangka input biaya (TextInputLayout).

Line 40: Baris kosong.

Line 41 - 46: Membuka <TextInputLayout> kedua untuk menu dropdown persentase tip. Atribut gayanya diubah menggunakan akhiran .ExposedDropdownMenu agar berfungsi sebagai menu pilihan. Atribut

`app:prefixText="% "` digunakan untuk menyisipkan simbol persen secara otomatis di dalam kotak.

Line 47: Baris kosong.

Line 48 - 53: Mendefinisikan `<AutoCompleteTextView>` yang diberi ID `dropdownTip`. Komponen ini bertindak sebagai kotak teks yang jika diklik akan memunculkan menu. Atribut `android:inputType="none"` memastikan pengguna tidak bisa mengetik manual di kotak ini.

Line 54: Penutup untuk blok kerangka dropdown (`TextInputLayout`).

Line 55: Baris kosong.

Line 56 - 61: Membuka tag `<LinearLayout>` baru dengan orientasi horizontal (baris 61). Komponen ini bertindak persis seperti Row di Jetpack Compose, digunakan untuk menyusun teks dan tombol geser secara menyamping (kiri ke kanan).

Line 62: Baris kosong.

Line 63 - 68: Mendefinisikan `<TextView>` untuk label "Round up tip?". Atribut `android:layout_weight="1"` (baris 66) adalah kunci untuk mendorong komponen Switch di sebelahnya agar mentok ke ujung kanan layar.

Line 69: Baris kosong.

Line 70 - 73: Mendefinisikan komponen `<SwitchMaterial>` yang diberi ID `switchRoundUp`. Ini adalah tombol geser yang dapat diubah statusnya (nyala/mati) untuk opsi pembulatan nilai.

Line 74: Penutup untuk blok `LinearLayout` horizontal.

Line 75: Baris kosong.

Line 76 - 82: Mendefinisikan `<TextView>` terakhir yang diberi ID `tvTipAmount`. Komponen ini berfungsi menampilkan hasil akhir perhitungan dengan ukuran huruf besar (36sp) dan dicetak tebal (bold). (Catatan: Simbol = di akhir baris 82 pada kodemu adalah salah ketik (typo) yang sebaiknya dihapus di Android Studio agar tidak memicu peringatan).

Line 83: Baris kosong.

Line 84: Penutup untuk blok susunan vertikal utama (`LinearLayout`).

Line 85: Penutup untuk blok terluar layar (`ScrollView`).

Pembahasan Compose

MainActivity.kt

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul2compopse`. Ini menentukan identitas dari project aplikasi versi Compose kamu.

Line 2: Baris kosong sebagai pemisah.

Line 3 - 38: Blok import yang memasukkan komponen dan fungsi bawaan dari library Jetpack Compose (seperti `Column`, `Text`, `TextField`, `mutableStateOf`, dll). Kita juga mengimpor `ComponentActivity` sebagai kelas dasar untuk aplikasi Compose, serta library `NumberFormat` untuk memformat mata uang.

Line 39 - 40: Baris kosong.

Line 41: Mendeklarasikan class `MainActivity` yang mewarisi (`extends`) `ComponentActivity()`. Ini adalah kelas dasar standar untuk aplikasi Android modern yang menggunakan Jetpack Compose.

Line 42: Meng-override fungsi `onCreate`. Ini adalah titik awal (entry point) di mana siklus hidup aplikasi berjalan saat pertama kali dibuka.

Line 43: Memanggil `super.onCreate` agar sistem Android memproses pembuatan activity bawaannya.

Line 44: Memanggil `enableEdgeToEdge()` agar tampilan aplikasi bisa memanjang penuh ke ujung layar (menembus area status bar di atas dan navigasi di bawah).

Line 45: `setContent {}` adalah fungsi krusial di Compose yang bertugas mendefinisikan tampilan UI (layout) langsung di dalam file Kotlin ini, menggantikan fungsi `setContentView(R.layout...)` pada arsitektur XML.

Line 46: Memanggil tema bawaan aplikasi yaitu `Modul2CompopseTheme`.

Line 47 - 50: Mendefinisikan `Surface` yang bertindak sebagai kanvas latar belakang utama aplikasi, dengan ukuran memenuhi layar (`fillMaxSize`) dan warna sesuai tema sistem.

Line 51: Memanggil fungsi `TipCalculatorApp()` yang berisi seluruh komponen antarmuka aplikasi buatan kita.

Line 52 - 56: Penutup kurung kurawal untuk blok `Surface`, `Theme`, `setContent`, fungsi `onCreate`, dan kelas `MainActivity`.

Line 57: Baris kosong.

Line 58: Anotasi `@OptIn(ExperimentalMaterial3Api::class)` ditambahkan karena komponen dropdown yang kita gunakan masih berstatus eksperimental di dalam library Material 3 Compose.

Line 59: Anotasi `@Composable` yang menandakan bahwa fungsi di bawahnya bukan fungsi Kotlin biasa, melainkan fungsi yang bertugas menggambar komponen antarmuka (UI) ke layar.

Line 60: Mendeklarasikan fungsi utama tampilan bernama `TipCalculatorApp()`.

Line 61 - 62: Membuat variabel state bernama `amountInput` menggunakan `mutableStateOf("")` dan `remember`. State ini bertugas mengingat dan menyimpan teks (biaya) yang diketik oleh pengguna.

Line 63: Baris kosong.

Line 64 - 67: Mendeklarasikan variabel state untuk fitur dropdown. `tipInput` untuk menyimpan angka tip yang dipilih (default "15"), `expanded` untuk menyimpan status menu terbuka/tertutup (default false), dan `tipOptions` yang berisi daftar pilihan persentase ("15", "18", "20").

Line 68: Baris kosong.

Line 69: Membuat variabel state bernama `roundUp` untuk menyimpan status tombol switch pembulatan, dengan nilai awal false (mati).

Line 70: Baris kosong.

Line 71 - 72: Mengekstrak teks dari `amountInput` dan `tipInput`, lalu mengubahnya menjadi angka desimal (Double). Jika kosong/tidak valid, kembalikan nilai 0.0 dengan operator elvis (`?:`).

Line 73: Baris kosong.

Line 74: Memanggil fungsi `calculateTip()` dengan memasukkan variabel angka dari baris sebelumnya, lalu menyimpan hasil format akhirnya ke dalam variabel `tip`. Baris 71-74 ini akan dieksekusi ulang secara otomatis setiap kali ada state yang berubah (Recomposition).

Line 75: Baris kosong.

Line 76 - 82: Membuka komponen `Column` yang bertugas menyusun seluruh elemen di dalamnya secara vertikal (dari atas ke bawah). Modifier digunakan untuk memberi padding 40.dp, mengaktifkan fitur `verticalScroll` agar layar bisa digeser, dan menarik seluruh isinya agar rata tengah (`CenterHorizontally` dan `Center`).

Line 83 - 89: Mendefinisikan komponen Text untuk menampilkan judul "Calculate Tip", dengan tambahan modifier padding bawah dan ditarik rata kiri (Alignment.Start).

Line 90: Baris kosong.

Line 91 - 110: Mendefinisikan komponen TextField sebagai kotak input biaya. Di baris 93-97 (onValueChange), terdapat logika validasi untuk mencegah pengguna memasukkan huruf atau simbol selain angka desimal. Baris 99-104 menambahkan leadingIcon berupa gambar pensil (Icons.Default.Edit). Atribut keyboardOptions memaksa memunculkan keyboard angka.

Line 111 - 112: Baris kosong dan komentar.

Line 113 - 117: Membuka komponen ExposedDropDownMenuBox sebagai wadah utama fitur dropdown. Atribut onExpandedChange mengatur agar status terbuka/tertutup (expanded) terbalik (toggle) setiap kali kotak ini diklik.

Line 118 - 134: Mendefinisikan TextField sekunder yang dirancang khusus hanya untuk diklik (bukan diketik). Parameter readOnly = true mencegah munculnya keyboard. Teks yang ditampilkan disisipkan simbol "%", dan ada leadingIcon teks "%" serta trailingIcon (ikon panah bawah/atas otomatis) di bagian kanan.

Line 135 - 138: Membuka komponen ExposedDropDownMenu yang bertindak sebagai menu melayang tempat daftar pilihan muncul.

Line 139 - 148: Melakukan perulangan (forEach) pada variabel tipOptions. Untuk setiap angka di dalamnya, Compose mencetak komponen DropdownMenuItem. Jika opsi ini diklik (onClick), angka akan disimpan ke tipInput dan menu langsung ditutup (expanded = false).

Line 149 - 150: Penutup kurung kurawal untuk ExposedDropDownMenuBox dan baris kosong.

Line 151 - 157: Membuka komponen Row untuk menyusun teks dan tombol switch secara menyamping (horizontal). Atribut horizontalArrangement = Arrangement.SpaceBetween digunakan untuk mendorong komponen ke sisi paling kiri dan paling kanan.

Line 158: Mendefinisikan Text "Round up tip?".

Line 159 - 162: Mendefinisikan komponen Switch. Nilai checked-nya diikat pada state roundUp, dan onCheckedChange akan memperbarui state tersebut jika switch digeser pengguna.

Line 163 - 164: Penutup kurung kurawal untuk Row dan baris kosong.

Line 165 - 168: Mendefinisikan Text untuk menampilkan hasil akhir. Parameter text disematkan variabel \$tip yang nilai matematikanya akan selalu diperbarui secara otomatis. Gaya huruf dibesarkan menggunakan MaterialTheme.typography.displaySmall.

Line 169 - 170: Penutup kurung kurawal untuk komponen Column dan fungsi TipCalculatorApp().

Line 171: Baris kosong.

Line 172: Mendeklarasikan fungsi tambahan bernama calculateTip() yang bersifat private. Fungsi ini menerima nominal, persentase, dan status pembulatan untuk memproses logika matematika aplikasi.

Line 173 - 174: Menghitung persentase dasar dengan membagi tipPercent dengan 100, lalu mengalikannya dengan total biaya (amount).

Line 175 - 176: Baris kosong.

Line 177 - 179: Membuka kondisional if. Jika roundUp bernilai true, nilai tip dibulatkan ke atas menggunakan kotlin.math.ceil().

Line 180: Baris kosong.

Line 181 - 182: Memformat nilai tip menjadi bentuk mata uang (berisi simbol dan desimal rapi) menggunakan getCurrencyInstance().format(), lalu mengembalikannya sebagai output fungsi untuk ditampilkan di layar.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Perbedaan utama antara implementasi XML (View System tradisional) dan Jetpack Compose (modern) terletak pada pergeseran paradigma dari *imperative* ke *declarative*. Dalam pendekatan XML, pengembang mendefinisikan "bagaimana" UI harus berubah secara manual, sedangkan dalam Jetpack Compose, pengembang mendeskripsikan "apa" yang harus ditampilkan berdasarkan status (*state*) saat ini.

Tabel 4 Perbandingan Jawaban Soal 2

Aspek Perbandingan	XML (Traditional View System)	Jetpack Compose (Modern Toolkit)
Paradigma	Imperative: Pengembang mengelola perubahan UI secara manual menggunakan setter (misal: <code>setText()</code>).	Declarative: UI diperbarui secara otomatis melalui proses recomposition saat data (<i>state</i>) berubah.
Lokasi Layout	Terpisah dalam berkas <code>.xml</code> di folder <code>res/layout</code> .	Terpadu dalam berkas Kotlin (<code>.kt</code>) menggunakan fungsi <code>@Composable</code> .
Kode Boilerplate	Sangat banyak (memerlukan <code>findViewById</code> atau <code>Binding</code> untuk mengakses UI).	Sangat minim; interaksi antara logika dan UI jauh lebih ringkas.
Prinsip UI	Berbasis pewarisan (Inheritance) dari kelas <code>View</code> .	Berbasis komposisi (<code>Composition</code>) fungsi-fungsi kecil.
Waktu Pengembangan	Cenderung lebih lama karena harus mengelola sinkronisasi antara XML dan Kotlin.	Lebih cepat karena kode lebih sedikit dan manajemen <i>state</i> lebih efisien.

Kelebihan dan Kekurangan

1. XML (Traditional View System)

- Kelebihan: Memiliki ekosistem yang sudah sangat matang, dokumentasi yang melimpah selama bertahun-tahun, dan didukung oleh alat desain visual (Layout Editor) yang stabil.
- Kekurangan: Sulit untuk mengelola UI yang sangat dinamis, rentan terhadap kesalahan sinkronisasi data-tampilan, dan memerlukan banyak baris kode hanya untuk tugas sederhana.

2. Jetpack Compose

- Kelebihan: Menghilangkan pemisahan antara layout dan logika, mempermudah pembuatan UI yang kompleks secara dinamis, dan secara signifikan mengurangi potensi *bug* terkait pembaruan manual UI.
- Kekurangan: Memerlukan pemahaman konsep baru tentang *recomposition* dan *state management*, serta masih dalam tahap pengembangan aktif untuk beberapa fitur tingkat lanjut.

SOAL 3

apa saja yang jadi daya jual Kotlin. Contoh (null safety, chain, elvish)

A. Pembahasan

Kotlin memiliki berbagai keunggulan teknis yang menjadikannya bahasa pilihan utama dalam pengembangan perangkat lunak modern, khususnya pada platform Android. Fitur-fitur ini tidak hanya meningkatkan produktivitas pengembang, tetapi juga secara signifikan mengurangi potensi kesalahan dalam kode.

- **Null Safety:** Salah satu kontribusi terbesar Kotlin adalah eliminasi kesalahan *NullPointerException* (NPE) melalui sistem tipenya. Kotlin membedakan antara referensi yang boleh bernilai *null* (String?) dan yang tidak boleh (*non-nullable*), sehingga potensi *crash* dapat dideteksi sejak tahap kompilasi.
- **Elvish Operator (?:):** Operator ini memberikan cara yang sangat ringkas untuk menangani nilai *null* dengan menyediakan nilai cadangan (*default*). Sebagai contoh, dalam kalkulator tip, jika input pengguna tidak valid, pengembang dapat menggunakan `?: 0.0` untuk memastikan aplikasi tetap berjalan dengan nilai nol.
- **Safe Call Operator (?.):** Memungkinkan pengembang melakukan pemanggilan properti atau fungsi pada objek yang mungkin bernilai *null*. Jika objek tersebut ternyata kosong, sistem tidak akan memicu *error*, melainkan mengembalikan nilai *null* secara aman.
- **Ringkas dan Ekspresif:** Kotlin mampu mengurangi jumlah baris kode (*boilerplate code*) secara signifikan dibandingkan bahasa Java melalui fitur seperti *Data Classes* dan *Inference Types*. Hal ini membuat kode lebih mudah dibaca dan dipelihara dalam jangka panjang.
- **Interoperabilitas Penuh:** Kotlin dirancang agar dapat bekerja secara berdampingan dengan kode Java. Artinya, pengembang dapat menggunakan library Java yang sudah ada di dalam proyek Kotlin tanpa hambatan teknis.
- **Scope Functions & Chaining:** Kotlin menyediakan fungsi-fungsi seperti `.let`, `.apply`, dan `.also` yang mendukung penulisan kode secara berantai (*chaining*). Fitur ini mempermudah konfigurasi objek dan pengolahan data dalam urutan yang linear dan bersih.

DAFTAR PUSTAKA

- Singh, R., Rohit, E., Singh, K., & Singh, A. (2025). Comparative study of XML vs Jetpack Compose for UI Development in Android. *International Journal of Computer Science and Information Security*, 12(1), 6175–6184.
- Talib, M. S., & Hassan, A. H. (2024). Kotlin Programming Language: A Comprehensive Overview of Definition, Applications, Advantages, and Limitations. *International Journal of Computer Applications Technology and Research*, 13(03), 7–9.

TAUTAN GIT

<https://github.com/Ryuzora/praktikum-mobile/tree/main/Modul2>